Software Technology 08

Design Patterns II.

Design Pattern Types

- Creational
- Structural
- Behavioral
- Concurrency
- Architectural
- Distributed
- Algorithm Strategy
- Implementation Strategy

Concurrency Design Patterns

Monitor Object

- Mutual exclusion
- For access to shared resources:
  - Memory, cache, files etc.
- Spin-Lock: check lock repeatedly until acquired (busy waiting)
- Lock: sleep until lock acquired or timeout
- Condition variable: sleep until another thread’s signal
- Java’s *synchronized* keyword

Spin lock

```
locked:                         ; The lock variable. 1 = locked, 0 = unlocked.
     dd      0

spin_lock:
     mov     eax, 1             ; Set the EAX register to 1.
     xchg    eax, [locked]      ; Atomically swap the EAX register with
                                ;  the lock variable.
                                ; This will always store 1 to the lock, leaving
                                ;  the previous value in the EAX register.
     test    eax, eax           ; Test EAX with itself. Among other things, this will
                                ;  set the processor's Zero Flag if EAX is 0.
                                ; If EAX is 0, then the lock was unlocked and
                                ;  we just locked it.
                                ; Otherwise, EAX is 1 and we didn't acquire the lock.
     jnz     spin_lock          ; Jump back to the MOV instruction if the Zero Flag is
                                ;  not set; the lock was previously locked, and so
                                ; we need to spin until it becomes unlocked.
     ret                        ; The lock has been acquired, return to the calling
                                ;  function.

spin_unlock:
     xor     eax, eax           ; Set the EAX register to 0.
     xchg    eax, [locked]      ; Atomically swap the EAX register with
                                ;  the lock variable.
     ret                        ; The lock has been released.
```

Lock

```
#include <chrono>
#include <iostream>
#include <mutex>
#include <thread>

int g_num = 0; // protected by g_num_mutex
std::mutex g_num_mutex;

void slow_increment(int id)
{
    for (int i = 0; i < 3; ++i)
    {
        g_num_mutex.lock();
        ++g_num;
        // note, that the mutex also syncronizes the output
        std::cout << "id: " << id << ", g_num: " << g_num << '\n';
        g_num_mutex.unlock();

        std::this_thread::sleep_for(std::chrono::milliseconds(234));
    }
}

int main()
{
    std::thread t1{slow_increment, 0};
    std::thread t2{slow_increment, 1};
    t1.join();
    t2.join();
}
```

Condition variable

```cpp
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <string>
#include <thread>

std::mutex m;
std::condition_variable cv;
std::string data;
bool ready = false;
bool processed = false;

void worker_thread()
{
    // Wait until main() sends data
    std::unique_lock lk(m);
    cv.wait(lk, []{ return ready; });

    // after the wait, we own the lock.
    std::cout << "Worker thread is processing data\n";
    data += " after processing";

    // Send data back to main()
    processed = true;
    std::cout << "Worker thread signals data processing completed\n";

    // Manual unlocking is done before notifying, to avoid waking up
    // the waiting thread only to block again (see notify_one for details)
    lk.unlock();
    cv.notify_one();
}
```

```cpp
int main()
{
    std::thread worker(worker_thread);

    data = "Example data";
    // send data to the worker thread
    {
        std::lock_guard lk(m);
        ready = true;
        std::cout << "main() signals data ready for processing\n";
    }
    cv.notify_one();

    // wait for the worker
    {
        std::unique_lock lk(m);
        cv.wait(lk, []{ return processed; });
    }
    std::cout << "Back in main(), data = " << data << '\n';

    worker.join();
}
```

Java: *synchronized*

```java
public class SynchronizedMethods {

    private int sum = 0;

    public void calculate() {
        setSum(getSum() + 1);
    }

    // standard setters and getters
}
```

```java
@Test
public void givenMultiThread_whenNonSyncMethod() {
    ExecutorService service = Executors.newFixedThreadPool(3);
    SynchronizedMethods summation = new SynchronizedMethods();

    IntStream.range(0, 1000)
      .forEach(count -> service.submit(summation::calculate));
    service.awaitTermination(1000, TimeUnit.MILLISECONDS);

    assertEquals(1000, summation.getSum());
}
```

```java
public synchronized void synchronisedCalculate() {
    setSum(getSum() + 1);
}
```

- Synchronized method:
  - synchronized over the instance
- Synchronized static method:
  - synchronized over the class object

Active Object

- Goal: Method invocation and execution on separate threads
- Asynchronous method invocation:
  return immediately, get result later
- Active object usually has its own thread of control
- Elements:
  - Proxy: Client interface; Methods just put tasks into a queue
  - Scheduler: Selects next task from the queue
  - Implementation: Runs the task
  - Getting result:
    - Callback
    - Variable to hold result later: Future, Promise

Future

```java
public class SquareCalculator {

    private ExecutorService executor
      = Executors.newSingleThreadExecutor();

    public Future<Integer> calculate(Integer input) {
        return executor.submit(() -> {
            Thread.sleep(1000);
            return input * input;
        });
    }
}
```

```java
Future<Integer> future = new SquareCalculator().calculate(10);

while(!future.isDone()) {
    System.out.println("Calculating...");
    Thread.sleep(300);
}

Integer result = future.get();
```

```java
Integer result = future.get(500, TimeUnit.MILLISECONDS);
```

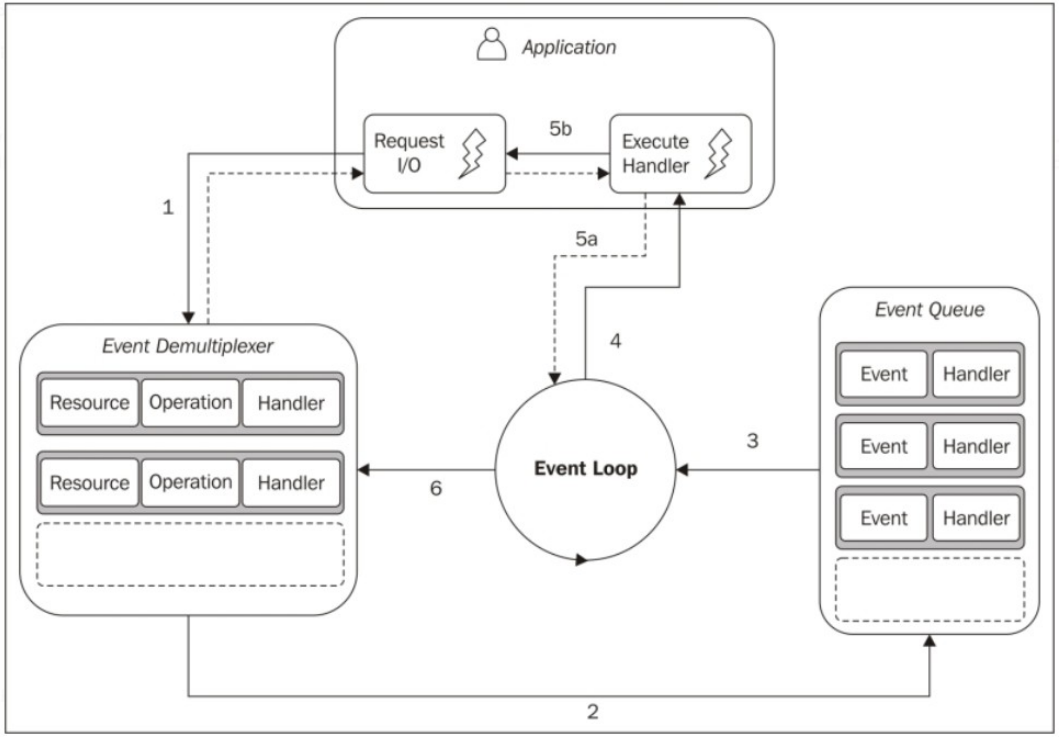
Reactor: Motivation

- Problem to solve:
  - Receiving or sending data via network connections (reading from or writing to buffers)
  - CPU is faster than the network: empty/full buffers

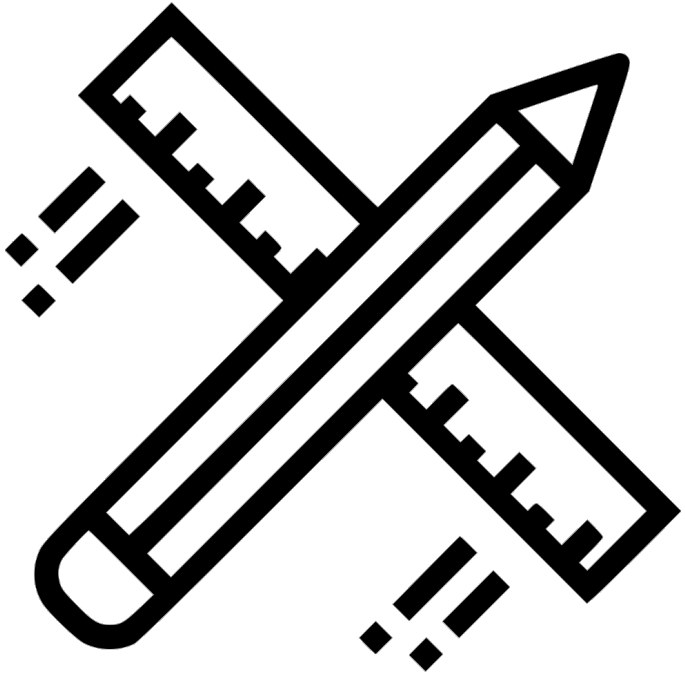
    → waiting
- Sub-optimal solution: dedicated thread for each connection
  - Waiting is not a problem any more
  - What if there are many millions of connections?

Reactor: Solution

- Selective waiting: “Is there a buffer that I can work with?”
  - Called “synchronous event demultiplexing”
  - The result can be used without waiting
  - A single thread is enough, running an *event loop*
- Handlers to offload heavy business logic to separate threads
- Inverted control flow (dispatch depends on arrived event)



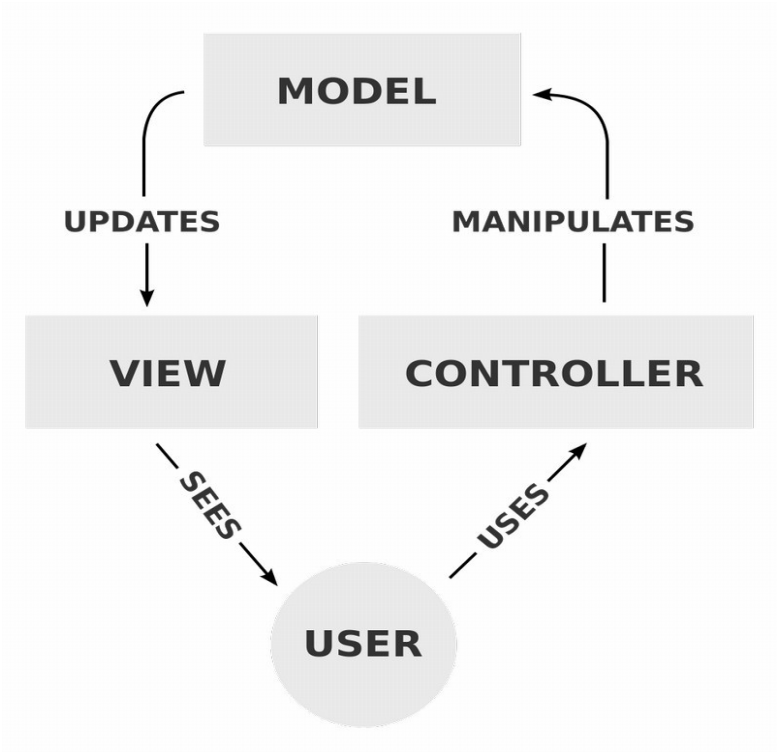
Architectural Design Patterns



MVC

– *Model-View-Controller*

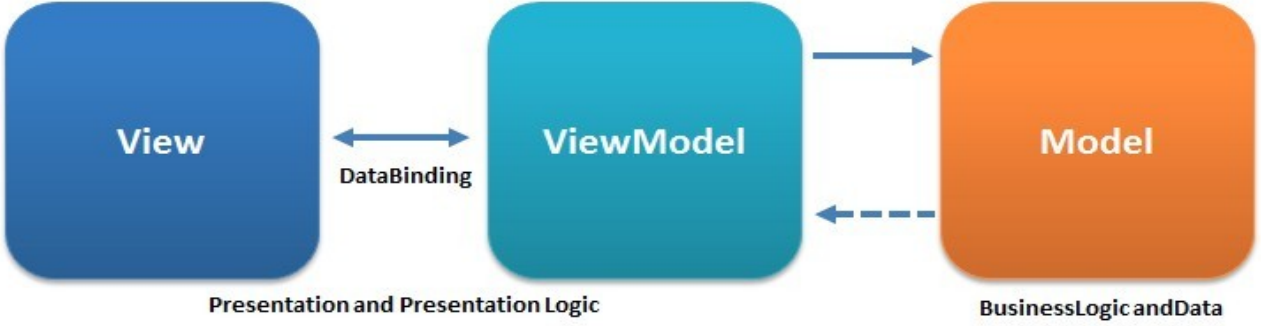
- GUI, Web, Game...



MVVM

– *Model-View-ViewModel*

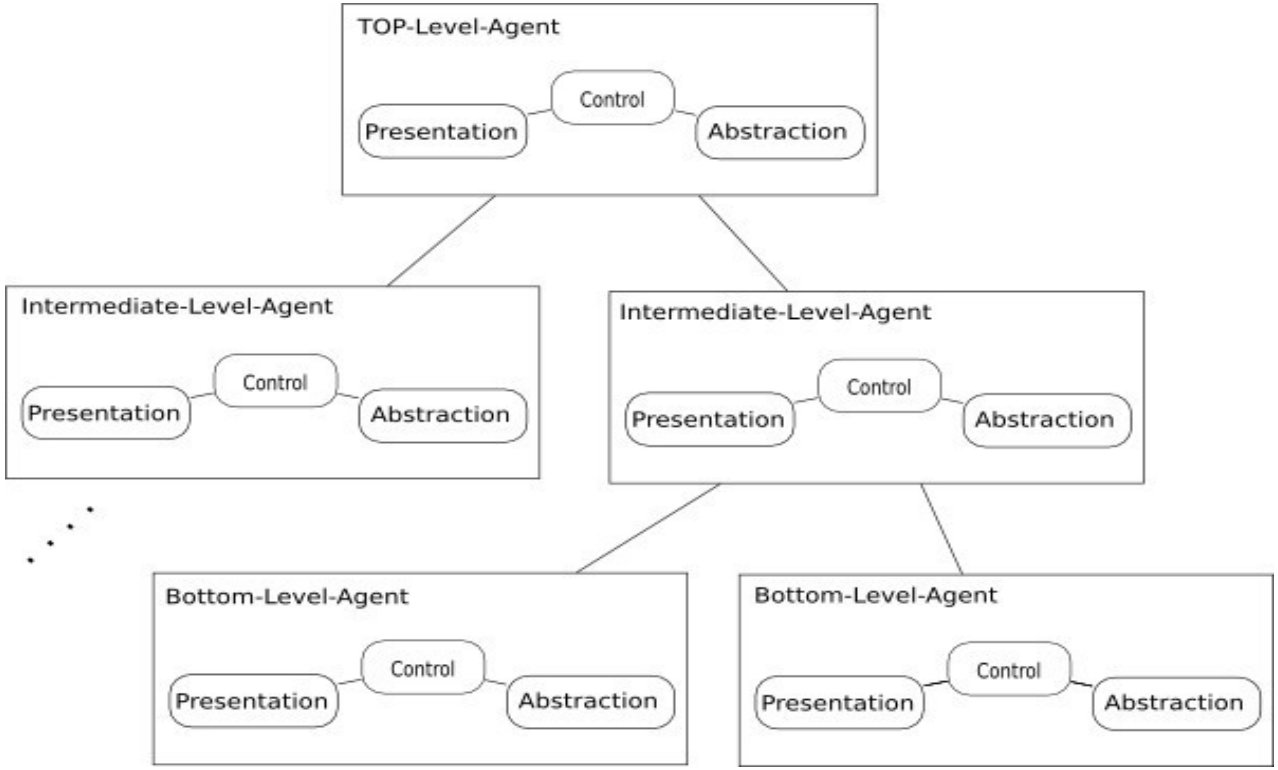
- Motivation:
  Separate presentation (View) from business logic (Model)
- ViewModel: Data binder or Mediator



PAC

– *Presentation-Abstraction-Control*

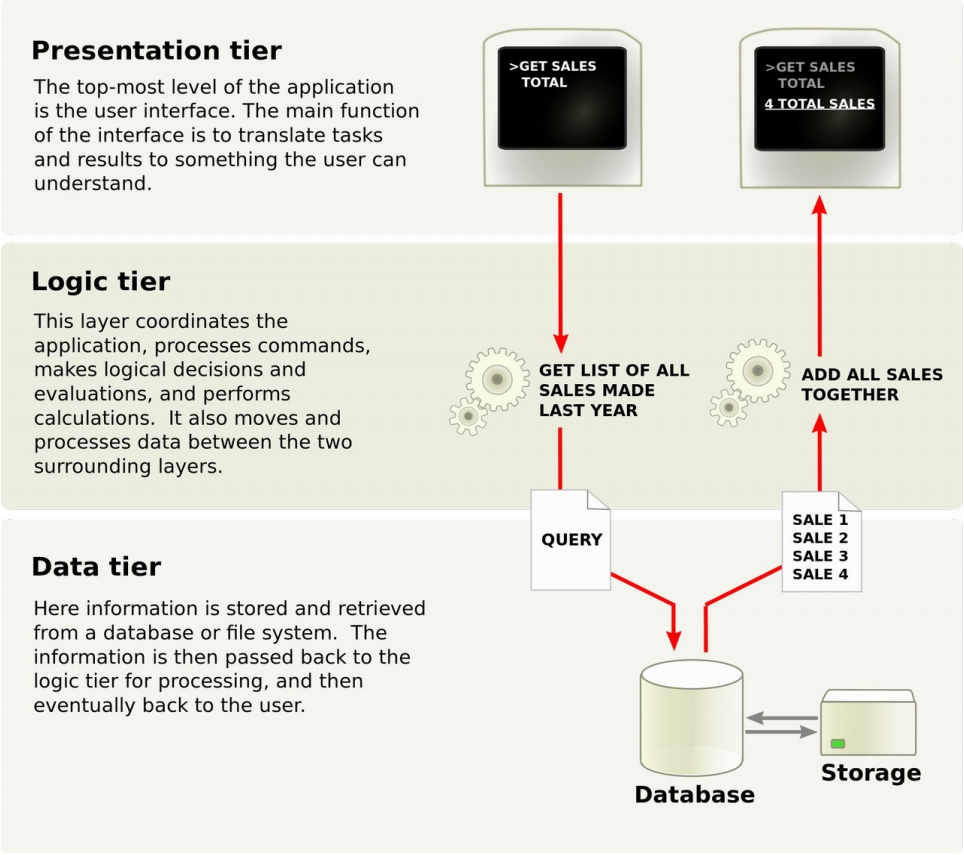
- Hierarchical MVC
- Blocks communicate only through Control objects



Multitier

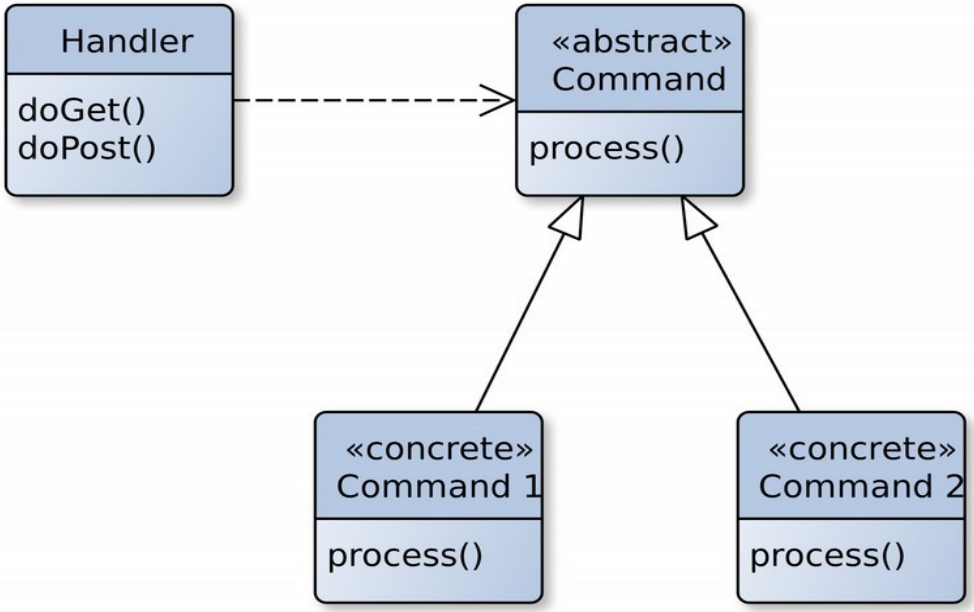
or N-tier

- Reduce complexity by separating system into tiers (layers)
- MVC variants are 3-tier



Front Controller

- Handles all requests to the system
- Serves as only entry-point for requests
- *index.php*



SOA

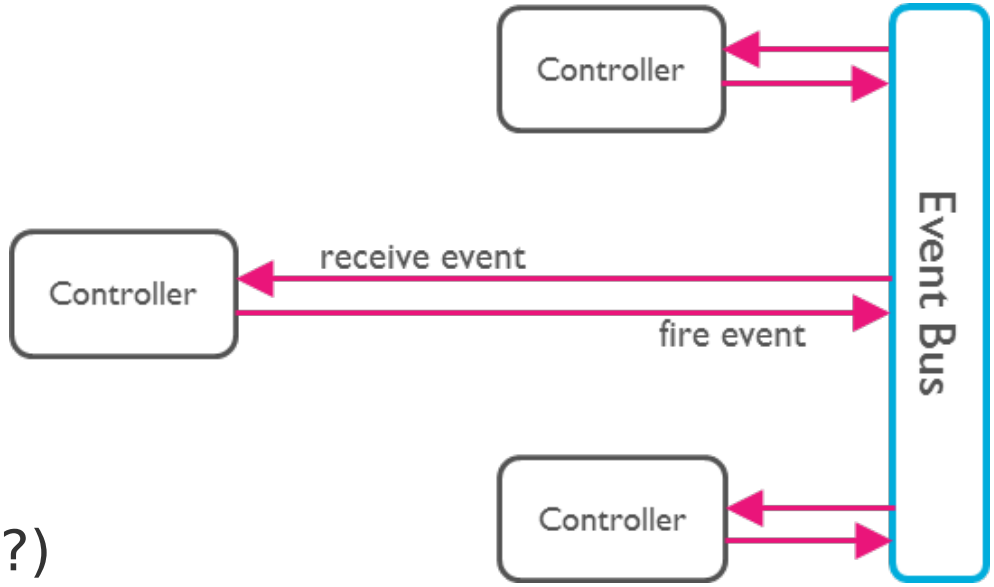
– *Service Oriented Architecture*

- Autonomous services (application components)
- Communicate through (network) protocol
- Find each other via registry (repository or broker)
- Can be hierarchical (service in service)
- Implementation agnostic

- Flexible, scalable
- Complex to maintain and test

Publish-Subscribe

- Event Bus
- Message queues
- Registering to messages via
  - Message types or Topics
  - Subscriber filtering
- the big bad Event System
- Against Coupling (good or bad?)



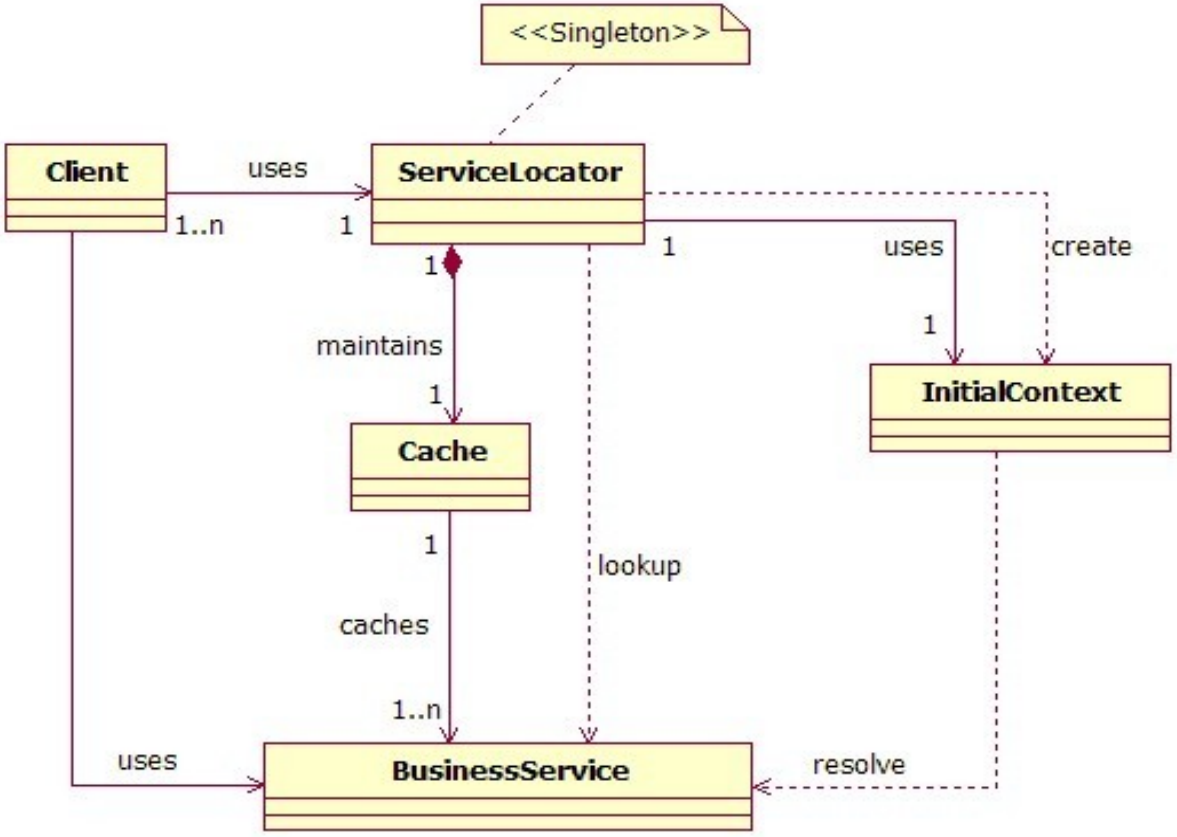
REST

– *Representational State Transfer*

- Not a standard, just a pattern
- Access and manipulate text through web
- URIs
- Uniform interface
  - input HTTP GET, POST, PUT, DELETE
  - output JSON, XML, HTML...

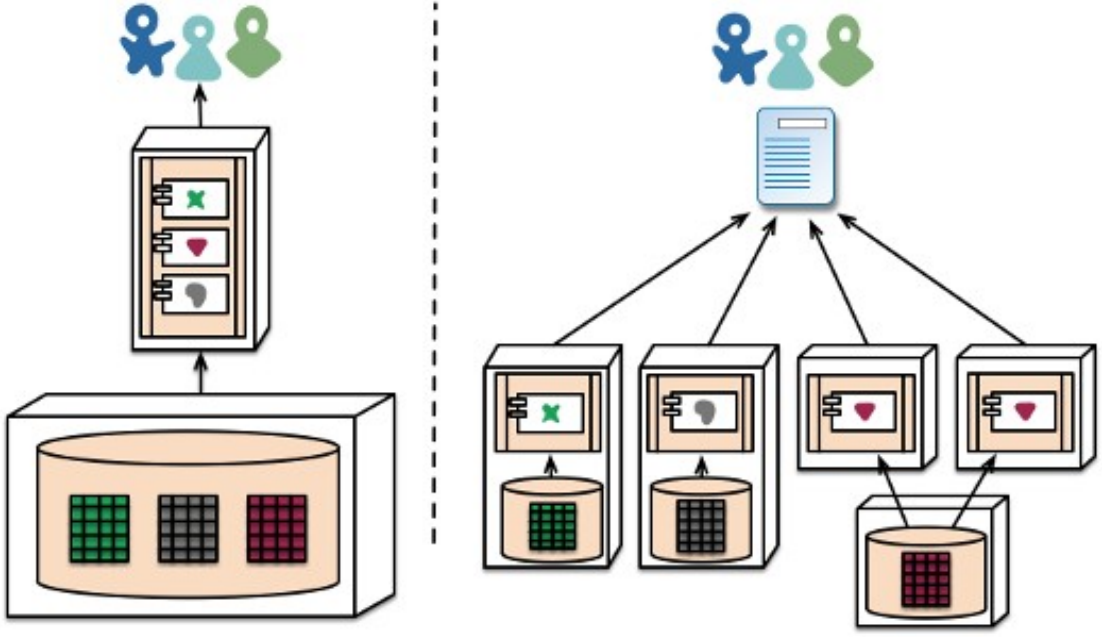
Service Locator

- Provides access information about other services



Microservices

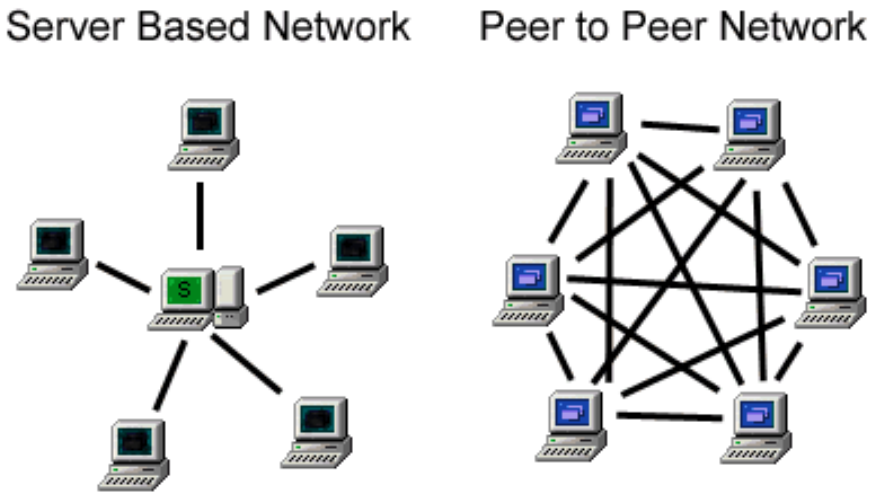
- = SOA + DevOps
- Small services
- Full automation of tests and deployment
- + automate
  - Fault tolerance
  - Scalability



P2P

– *Peer-to-peer*

- No client-server
- Routing and peer discovery
- Distributed
  - Storage
  - Search
  - Processing



Dependency Injection

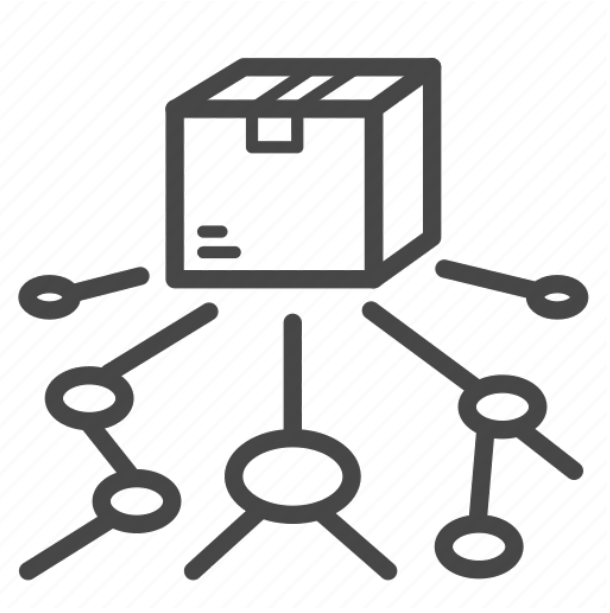
- Creator of object supplies (injects) all dependencies to the object
- Automate the construction of dependent services
- Separated usage and construction
- Through
  - Setter
  - Constructor
  - Automated, based on configuration (example: Guice)

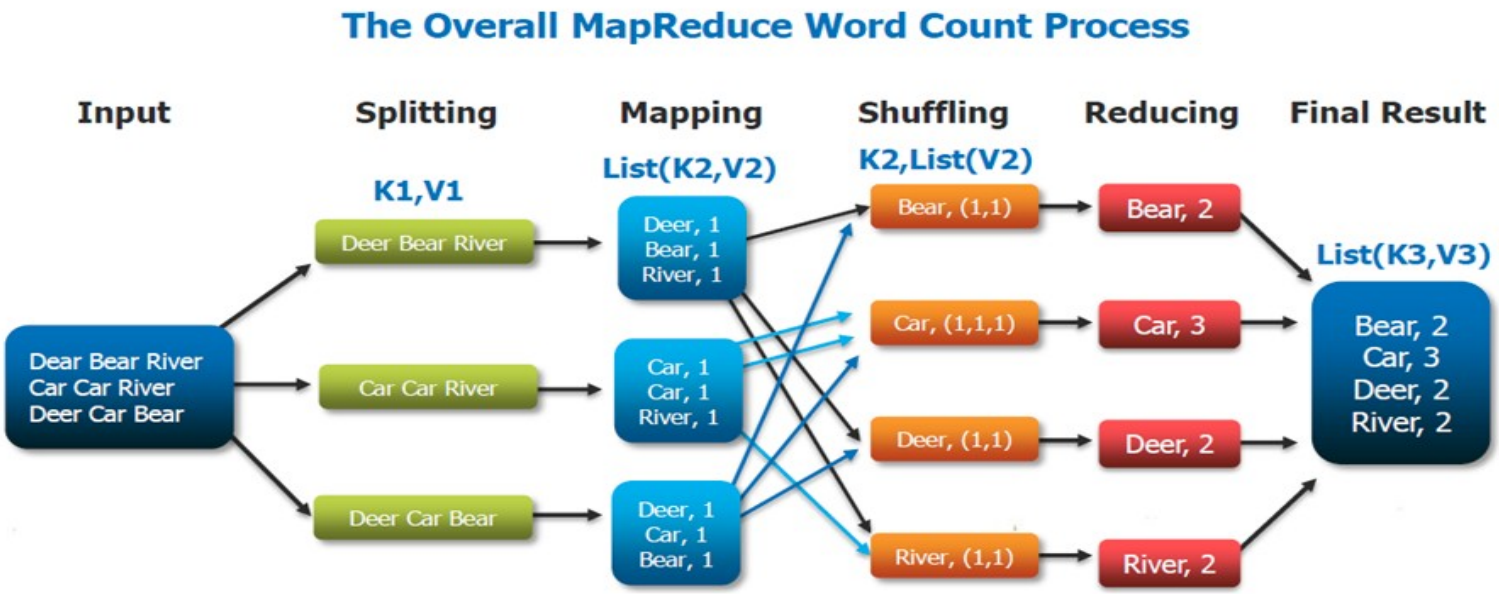
IoC

– *Inversion of Control*

- Normally custom code governs the program, calls general methods, libraries
- But a generic framework can also call specialized functions
- Examples:
  - Service Locator
  - Dependency Injection
  - Strategy
  - Template method

Distributed Design Patterns



MapReduce

- From functional programming
- Steps
  - Map
  - Shuffle
  - Reduce
- Examples:
  - Apache Hadoop
  - Spark